



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ
ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ Ακ. έτος 2024-
2025

Προχωρημένα Θέματα Βάσεων Δεδομένων

Ακαδημαϊκό έτος 2024-25, 9ο Εξάμηνο

Διδάσκων: Δημήτριος Τσουμάκος

Υπεύθυνος Εργαστηρίου: Νικόλαος Χαλβαντζής

Εξαμηνιαία Εργασία

Φοιτητές: Αναστασιάδης Βασίλειος ΑΜ:03119954

Θηβαίος Δημήτριος Ιωάννης ΑΜ:03119931

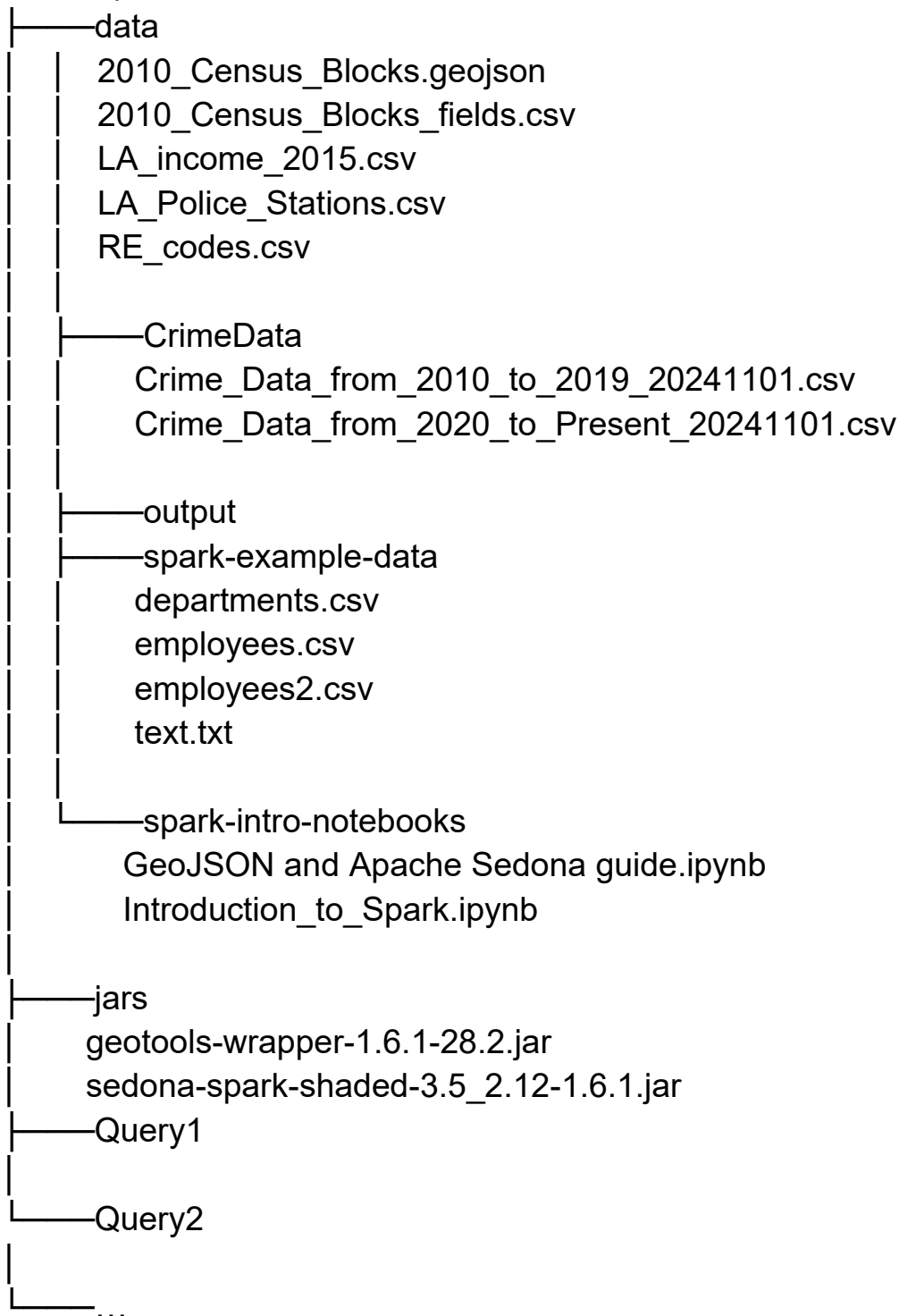
Το αποθετήριο στο οποίο είναι αποθηκευμένος ο κώδικας της εργασίας είναι το εξής:

https://github.com/ntua/advanced_db_project_group22

Για την εκτέλεση των queries απαιτείται τοπική χρήση του πυρήνα PySpark καθώς και των βιβλιοθηκών Apache Spark, Hadoop, Sedona, οδηγίες για την εκτέλεση αυτή παρέχονται στο αρχείο README.md του αποθετηρίου. Εκεί είναι διαθέσιμες μια σειρά εντολών η οποίες σε περιβάλλον Linux Ubuntu προχωράνε στην εγκατάσταση των απαραίτητων εργαλείων (python, Java, pyspark κτλ.).

Επιπλέον για την αυτόματη εκτέλεσή τους έχουν συμπεριληφθεί και σε ένα script αρχείο .sh

Μετά την εγκατάσταση των απαραίτητων εργαλείων είναι δυνατή η εκτέλεση των queries τοποθετώντας τα αρχεία δεδομένων csv στην εξής τοπολογία:



Query 1

Παρατηρήσαμε ότι η εκτέλεση του Query 1 με το DataFrame API ολοκληρώθηκε μέσα σε ~4 δευτερόλεπτα, ενώ η ίδια λογική με το RDD API χρειάστηκε περίπου 115 δευτερόλεπτα (σχεδόν 2 λεπτά). Αυτή η μεγάλη διαφορά οφείλεται κυρίως στο ότι το DataFrame API επωφελείται από εσωτερικές βελτιστοποιήσεις του Spark (όπως ο Catalyst Optimizer και το Project Tungsten), οι οποίες μειώνουν το κόστος διαχείρισης των δεδομένων και βελτιστοποιούν τις λειτουργίες φίλτρου, ομαδοποίησης και ταξινόμησης. Από την άλλη, το RDD API είναι πιο «χαμηλού επιπέδου», χωρίς επιπλέον βοήθεια από τον βελτιστοποιητή του Spark, με αποτέλεσμα να χρειάζεται περισσότερος χρόνος εκτέλεσης—ειδικά όταν ο όγκος των δεδομένων είναι μεγάλος.

Συνεπώς, σε συνθήκες όπως οι δικές μας (4 executors, αρκετά μεγάλου μεγέθους dataset), το DataFrame API υπερέχει σημαντικά σε απόδοση. Το RDD API μπορεί να δώσει ακριβώς τα ίδια αποτελέσματα, αλλά απαιτεί μεγαλύτερο χρόνο λόγω απουσίας βελτιστοποίησης. Γι' αυτό, στις περισσότερες περιπτώσεις, προτιμούμε το DataFrame API για μεγαλύτερη ταχύτητα και απλούστερη ανάπτυξη.

Query 2

Παρατηρήσαμε ότι η εκτέλεση με το DataFrame API ολοκληρώθηκε σε 12.00 δευτερόλεπτα, ενώ η χρήση του SQL API πέτυχε εξαιρετικά καλύτερους χρόνους εκτέλεσης, ολοκληρώνοντας το ίδιο query σε μόλις 3.19 δευτερόλεπτα (δηλαδή περίπου 3.8 φορές ταχύτερα). Αυτή η σημαντική διαφορά οφείλεται στο γεγονός ότι το SQL API επωφελείται από τις εσωτερικές βελτιστοποιήσεις του Catalyst Optimizer του Spark, ο οποίος εφαρμόζει τεχνικές όπως το Predicate Pushdown, το Column Pruning και το Join Optimization, μετατρέποντας το query σε μια βέλτιστη φυσική εκτέλεση και μειώνοντας έτσι τον χρόνο επεξεργασίας.

Αντίθετα, το DataFrame API προσφέρει μεγαλύτερο έλεγχο και ευελιξία μέσω Pythonic μεθόδων και είναι εξαιρετικά χρήσιμο για σύνθετες λειτουργίες και διαδικασίες ETL (Extract, Transform, Load), ωστόσο απαιτεί επιπλέον πόρους, αφού κάθε μετασχηματισμός περνά από επιπλέον φάσεις εκτέλεσης και βελτιστοποίησης μέσω του DAG (Directed Acyclic Graph).

Παράλληλα, στον τομέα της φόρτωσης και επεξεργασίας των δεδομένων, παρατηρήσαμε σημαντικές διαφορές μεταξύ των μορφών αποθήκευσης CSV και Parquet. Συγκεκριμένα, η φόρτωση των δεδομένων από CSV απαιτεί 5.74 δευτερόλεπτα, λόγω του ότι το CSV είναι μορφή απλού κειμένου που πρέπει να διαβαστεί και να αναλυθεί γραμμή-γραμμή, ενώ το Parquet, ως στήλη-βασισμένη μορφή, φορτώνει τα δεδομένα σε μόλις 0.10 δευτερόλεπτα χάρη στην αποδοτικότερη συμπίεση και την μείωση του όγκου των δεδομένων που χρειάζεται να ανακτηθεί.

Επιπλέον, όταν εκτελούνται SQL queries, τα δεδομένα στο Parquet επεξεργάζονται σε 2.12 δευτερόλεπτα, ενώ η ίδια διαδικασία με CSV ολοκληρώνεται σε 3.41 δευτερόλεπτα, δηλαδή περίπου 37% πιο αργά, λόγω της ανάγκης πλήρους σάρωσης των δεδομένων χωρίς την υποστήριξη προηγμένων τεχνικών όπως indexing και column pruning.

Query 3

Παρατηρώντας τα πλάνα εκτέλεσης και τους χρόνους που μετρήθηκαν, φαίνεται πως οι πιο αποδοτικές στρατηγικές join ήταν το Sort Merge Join (MERGE) και το Shuffle Hash Join (SHUFFLE_HASH), με χρόνους γύρω στα 12–13 δευτερόλεπτα. Αντιθέτως, το Broadcast Join χρειάστηκε περίπου 23 δευτερόλεπτα, γεγονός που υποδεικνύει ότι τα σύνολα δεδομένων δεν ήταν αρκετά μικρά για να ευνοηθεί η στρατηγική broadcast. Επίσης, το Shuffle Replicate NL (Nested Loop) είναι σχεδόν πάντα η λιγότερο αποδοτική επιλογή, οπότε ο Spark συνήθως καταλήγει σε sort-merge ή shuffle hash.

Σε γενικές γραμμές, για μεγάλα equi-join σενάρια:

- Sort Merge Join είναι η προεπιλογή του Spark όταν και τα δύο DataFrames έχουν αρκετό μέγεθος. Μετά το απαραίτητο shuffle και sort, η συγχώνευση (merge) γίνεται αρκετά αποτελεσματικά.
- Shuffle Hash Join μπορεί να είναι ακόμα πιο γρήγορη σε κάποιες περιπτώσεις, αρκεί να υπάρχει διαθέσιμη μνήμη και ο όγκος των δεδομένων δεν ξεπερνά τα όρια των executors.

Από την άλλη, το broadcast ενδείκνυται μόνο όταν το DataFrame που γίνεται broadcast χωράει εύκολα στη μνήμη (π.χ. χιλιάδες γραμμές, όχι εκατομμύρια). Διαφορετικά, το overhead του διαμοιρασμού ξεπερνά το όφελος. Τέλος, το nested loop (replicate NL) κάνει καρτεσιανό πολλαπλασιασμό και δύσκολα συμφέρει — εκτός αν μιλάμε για υπερβολικά μικρά δεδομένα ή δεν έχουμε equi-condition.

Επομένως, όπως έδειξαν και τα αποτελέσματά μας, Sort Merge και Shuffle Hash Join είναι οι καταλληλότερες επιλογές για τα μεγάλα δεδομένα του query, επιτρέποντας γρηγορότερη εκτέλεση σε σύγκριση με Broadcast ή Replicate NL.

Query 5

Παρατηρήσαμε ότι, σε μια σειρά από δοκιμές με διαφορετικές διαμορφώσεις, ο χρόνος εκτέλεσης κυμαινόταν ως εξής:

- Με 2 Executors × 4 Cores / 8GB Memory ολοκληρώθηκε σε 39.47 δευτερόλεπτα.
- Με 4 Executors × 2 Cores / 4GB Memory ολοκληρώθηκε σε 37.69 δευτερόλεπτα.
- Με 8 Executors × 1 Core / 2GB Memory ολοκληρώθηκε σε 33.67 δευτερόλεπτα.

Αυτά τα αποτελέσματα επιβεβαιώνουν ότι, όταν ο φόρτος εργασίας είναι εύκολα παραλληλοποιήσιμος – όπως στον υπολογισμό της απόστασης μεταξύ ενός εγκλήματος και ενός αστυνομικού τμήματος – η παραλληλοποίηση παίζει καθοριστικό ρόλο στην επίτευξη καλύτερης απόδοσης. Συγκεκριμένα, η διαμόρφωση με 8 Executors × 1 Core απέδωσε καλύτερα, καθώς επιτρέπει την ταυτόχρονη εκτέλεση περισσότερων ανεξάρτητων tasks, μειώνοντας έτσι τον συνολικό χρόνο επεξεργασίας. Από την άλλη, η επιλογή με 2 Executors × 4 Cores παρουσίασε τον μεγαλύτερο χρόνο εκτέλεσης, πιθανώς λόγω της συγκέντρωσης του φόρτου εργασίας σε λιγότερους executors, γεγονός που οδηγεί σε καθυστερήσεις.

Η λογική πίσω από τα αποτελέσματα είναι σαφής: για εργασίες που επιτρέπουν υψηλό βαθμό παραλληλοποίησης, η χρήση περισσότερων executors με λιγότερους πυρήνες ανά executor αποδεικνύεται πιο αποδοτική, καθώς επιτρέπει την εκτέλεση περισσότερων tasks ταυτόχρονα χωρίς σημαντική συμφόρηση στη CPU. Ωστόσο, αυτή η στρατηγική δεν είναι απαραίτητα καθολική· σε περιπτώσεις όπου τα tasks δεν είναι εύκολα παραλληλοποιήσιμα ή όταν το dataset είναι πολύ μεγαλύτερο, η αύξηση του αριθμού των executors μπορεί να προκαλέσει προβλήματα επικοινωνίας μεταξύ τους ή ακόμη και ζητήματα μνήμης (π.χ. OutOfMemoryError).

Συνεπώς, για τον συγκεκριμένο φόρτο εργασίας που βασίζεται σε ανεξάρτητους γεωχωρικούς υπολογισμούς, η διαμόρφωση με 8 Executors × 1 Core / 2GB Memory είναι η πιο αποδοτική, επιβεβαιώνοντας ότι σε τέτοιες περιπτώσεις η μεγιστοποίηση της παραλληλοποίησης οδηγεί σε καλύτερα αποτελέσματα.